

PHASE 2 · DEEP LEARNING FOR COMPUTER-AIDED DIAGNOSIS

Automated Malaria Cell Detection

Reproducing an EfficientNet-B0 ensemble on the NIH cell images exactly as the paper's code ran it, across all 10 folds, and planning a fairer test

AJ

J Amathziah

230139 · Kaggle runs,
repository

MM

C Murali Madhav

230115 · Code: notebook,
scripts

RY

Ravi Yadav

230131 · Slides

PJ

Pugazhendhi J

230066 · Report

What we did after Phase 1, and what we found



Phase 1 • reviewed 11 studies

Accuracy on the NIH cells is reported at 96–99.9 %, but most studies split images at random, so one patient's cells sit in both training and testing.



Phase 2 • reproduced the base paper

Marques et al. (2022): EfficientNet-B0 + a 10-fold ensemble. We rebuilt it from the authors' own code and trained all 10 folds.



Next • test it fairly

H1: does accuracy drop once slide-level leakage is removed? H2: does YUV stain normalisation win it back?

97.55 %

mean fold accuracy

paper 97.56 % · Δ -0.01 pp

97.62 %

10-model ensemble

paper 98.29 % · Δ -0.67 pp

99.73 %

ensemble ROC-AUC

paper 99.76 % · Δ -0.03 pp

≈ 11 min

per fold on an RTX 4090

paper ≈ 4 h 46 min per fold

All 10 folds within 1 percentage point of the paper. Every metric is computed from the saved predictions in `results/A0_10fold_paper/`.

PHASE 1

What the literature says

Eleven studies on automated malaria detection with the NIH cell images, 2016–2024

Why malaria diagnosis needs automation

The gold standard is manual microscopy of Giemsa-stained blood smears: a technician checks every cell for parasites. It works, but it doesn't scale.



Slow and laborious

Each slide holds hundreds of cells, counted one by one.



Expert-dependent

Needs a trained microscopist: scarce where malaria is common.



Subjective

Results vary between observers and labs for the same smear.

WHAT WE SET OUT TO BUILD

An automated deep-learning system that classifies single red-blood-cell images as parasitized or uninfected: accurate, consistent, fast, and light enough for point-of-care use.

Task binary classification of single-cell crops

Method deep CNNs, no hand-crafted features

Goal clinically reliable, deployable

NIH Malaria Cell Images: the field's common benchmark

27,558

single-cell images

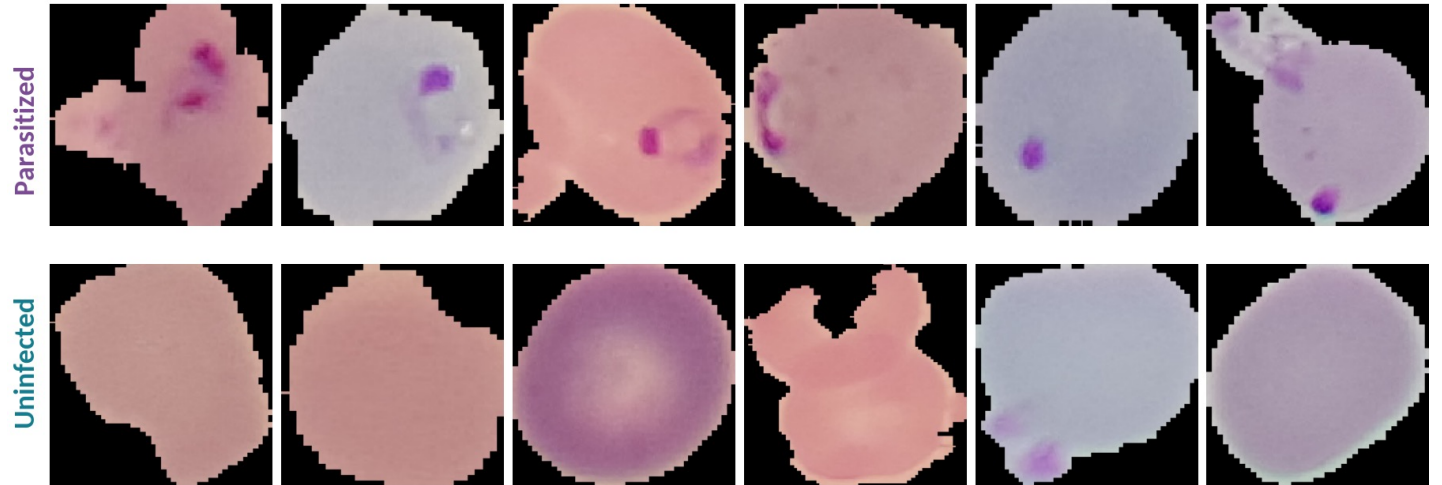
50 / 50

13,779 parasitized · 13,779 uninfected

200

patients · 150 infected, 50 healthy

Giemsa-stained thin blood smears from Chittagong Medical College Hospital, Bangladesh, segmented into single cells by the NIH.

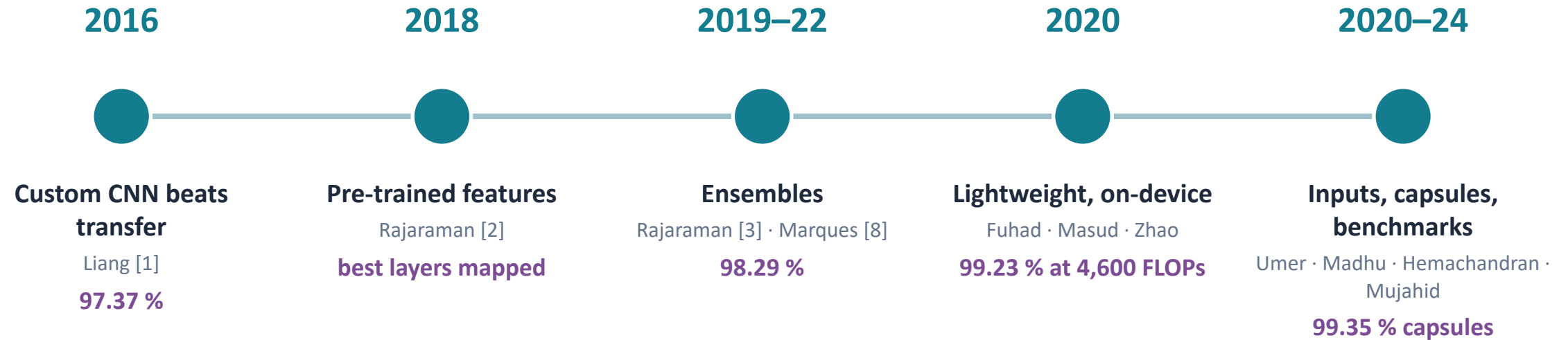


Infected cells show dark violet specks (the Giemsa stain). Crops are 40–394 px (median 130), all resized to 224 × 224.



Known issue, noisy labels: Fuhad et al. [5] found stain artifacts labelled as parasites and cleaned the set to 26,161 images. We use the standard 27,558 so our numbers compare with the paper.

How malaria CNNs evolved, 2016 → 2024



The arc: "can a CNN do this?" → squeeze accuracy from big models → shrink them for a phone → keep cell structure and benchmark fairly.

Five ideas that shaped the field



Domain features win

Liang [1]: a 16-layer CNN 97.37 % vs. AlexNet features 91.99 %. Rajaraman [2] mapped which pre-trained layers transfer best.



Ensembles steady answers

Rajaraman [3]: VGG-19 + SqueezeNet. Marques [8]: 10-fold EfficientNet-B0 ensemble, 98.29 %, AUC 99.76 %.



Small enough for a phone

Fuhad [5]: 99.23 % at 4,600 FLOPs, < 1 s on a phone. Masud [7]: 409 K parameters. Zhao [6]: 2 species.



Better inputs and structure

Umer [4]: YUV stain normalisation, ~99.9 %. Madhu [9]: capsule network keeps the parasite's pose, 99.35 %.



Fair benchmarks

Hemachandran [10]: MobileNetV2 97.06 % $\approx$ ResNet-50. Mujahid [11]: tuned 12-layer CNN 97.57 % beats 11 backbones.

Takeaway

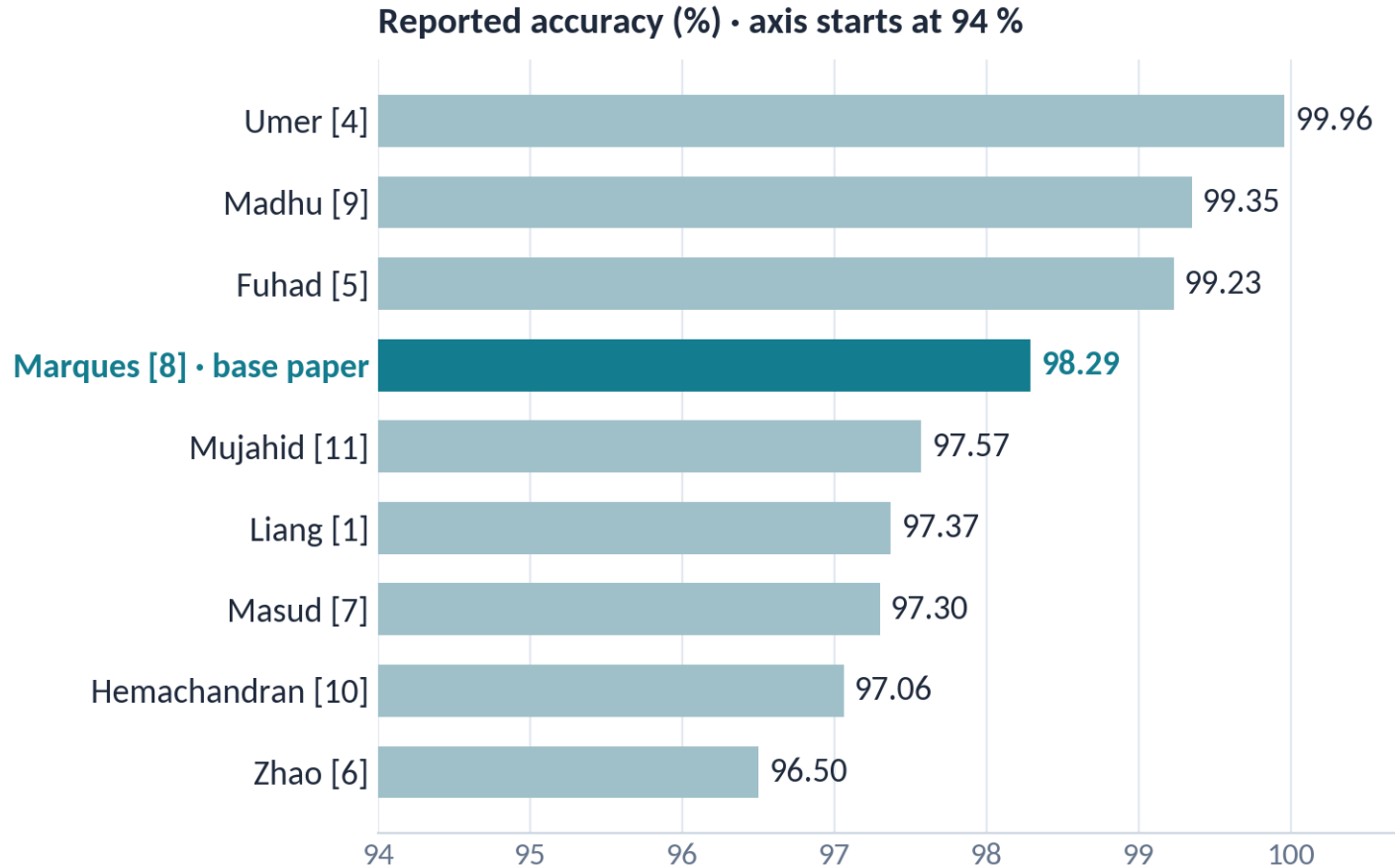
Accuracy on clean, randomly split data is largely solved. The open problems are trust (leakage, labels) and deployment.

All 11 studies on one page

#	Study (year)	Approach	Headline result
1	Liang (2016)	Custom 16-layer CNN from scratch	97.37 %, beats AlexNet features (91.99 %)
2	Rajaraman (2018)	Pre-trained CNNs as feature extractors	Mapped the best transfer layers
3	Rajaraman (2019)	VGG-19 + SqueezeNet ensemble	Lower variance, better generalisation
4	Umer (2020)	Stacked CNN + YUV stain normalisation	~99.9 % accuracy, 100 % precision (5-fold)
5	Fuhad (2020)	Autoencoder + knowledge distillation	99.23 % · 4,600 FLOPs · < 1 s · cleaned labels
6	Zhao (2020)	VGG16 + FSRCNN super-resolution + SSD300	96.5 % · cross-species (<i>P. vivax</i>)
7	Masud (2020)	Lightweight CNN + cyclical learning rate	97.30 % · 409 K parameters · ~97 min
8	Marques (2022) · base paper	EfficientNet-B0 · 10-fold ensemble	98.29 % · AUC 99.76 % · open code
9	Madhu (2023)	Inception-V3 + capsule network	99.35 % · preserves spatial pose
10	Hemachandran (2023)	CNN vs. MobileNetV2 vs. ResNet-50	MobileNetV2 97.06 % · light ≈ heavy
11	Mujahid (2024)	Custom 12-layer CNN vs. 11 backbones	97.57 % · lowest test loss (0.0995)

Direct ranking is misleading: protocols, splits and label cleaning differ between papers.

Reported accuracy is high, but not comparable



Different splits

5-fold, 10-fold and 80:20 protocols are not comparable.



Different data

Some use the noisy set, some cleaned labels.



Possible leakage

Random image splits can put one patient's cells in both training and test.

What none of the papers has solved



GAP 1

Patient-level data leakage

Random image-level splits put cells from the same patient in training and test, inflating accuracy. Only [1] and [2] split by patient.



GAP 2

Single-species focus

Almost all models see only *P. falciparum*; *vivax*, *ovale*, *malariae* and *knowlesi* are largely untested. Only Zhao [6] tried.



GAP 3

Noisy ground-truth labels

Except Fuhad [5], everyone trains and reports on the mislabelled public set.



GAP 4

Clinical-workflow gap

Work centres on clean, pre-segmented single cells; real labs need raw, unsegmented smears.

How the review shapes our project



YUV stain normalisation

from Umer [4]

Make the model robust to staining differences between labs. → H2



Patient-level validation

from Rajaraman [2]

Group cells by slide before splitting: honest, leakage-free scores. → H1



Verified labels

from Fuhad [5]

Learn real features, not artifacts (future work).



Lightweight model

from Masud [7] / Fuhad [5]

Keep inference light enough for low-resource settings.



First step: reproduce a strong, open baseline, then change one thing at a time. That is Phase 2.

PHASE 2

Reproducing the base paper

Marques et al. (2022), rebuilt from its own code and trained on all 10 folds

Why EfficientNet-B0 with a 10-fold ensemble



Open, executable code

The authors published their executed Jupyter notebook as a supplementary file, so a faithful rebuild is possible.



Strict validation

Stratified 10-fold cross-validation plus a separate hold-out set that no model trains on.



Light backbone

4.2 M parameters in total: the smallest EfficientNet, close to our deployability goal.



Strong, not at the ceiling

98.29 % leaves room to see whether a change helps or hurts.

THE PAPER'S TARGETS (from its own printed results)

97.56 %

mean fold accuracy (Table 5)

97.69 %

single model on hold-out (Table 6)

98.29 %

ensemble accuracy

99.76 %

ensemble ROC-AUC

First attempt: the recipe from the paper's text

Setting	First run (A0)
Hardware	Kaggle, 2× Tesla T4
Folds trained	3 of 10 (free GPU budget)
Weights	ImageNet EfficientNet-B0
Training	≤ 15 epochs, early stopping, batch 128
Augmentation	flips + 90° rotations
Hold-out set	10 % = 2,756 images

97.38 %**mean fold accuracy**

± 0.74 over 3 folds

97.57 %**3-model ensemble**

on 2,756 hold-out cells

99.76 %**ensemble ROC-AUC**

Within 1 pp on accuracy, but several settings were guesses because the text leaves them out. So we went to the authors' own code.

The authors' notebook differs from the paper's text in 8 places

Setting	Paper's code (used now)	Our first reproduction (from the text)
Pretrained weights	noisy-student · efficientnet 1.1.1 (qubvel)	ImageNet · tf.keras.applications
Classifier head	GAP → 128 → 64 → 32 → 2 softmax, dropout 0.3	GAP → dropout 0.2 → 1 sigmoid
Loss	categorical cross-entropy, label smoothing 0.1	binary cross-entropy
Schedule	33 epochs, LR × 0.5 on plateau, keep best val_loss	≤ 15 epochs, LR × 0.1, early stopping
Batch size	16 (1,240 steps per epoch)	128 (64 per GPU, 2 GPUs)
Hold-out set	20 % = 5,512 images	10 % = 2,756 images
Augmentation	full augmentations pipeline (noise, blur, distortions, CLAHE...)	flips + 90° rotations
Input	nearest-neighbour resize, pixels ÷ 255	bilinear resize, ImageNet normalisation

Source: Springer supplementary file MOESM1, the authors' executed notebook (30 Jan 2021). Our notebook now follows it line by line; 32 automated tests check the recipe.

Two things the paper's numbers hide



Evaluation images were augmented too

One ImageDataAugmentor object fed the training, test and validation generators, so every prediction the paper reports was made on a randomly distorted image.

We keep this for the headline numbers ("paper protocol") and also score the same models on clean images.



Precision and recall are swapped in the text

The paper's own printed ensemble report:

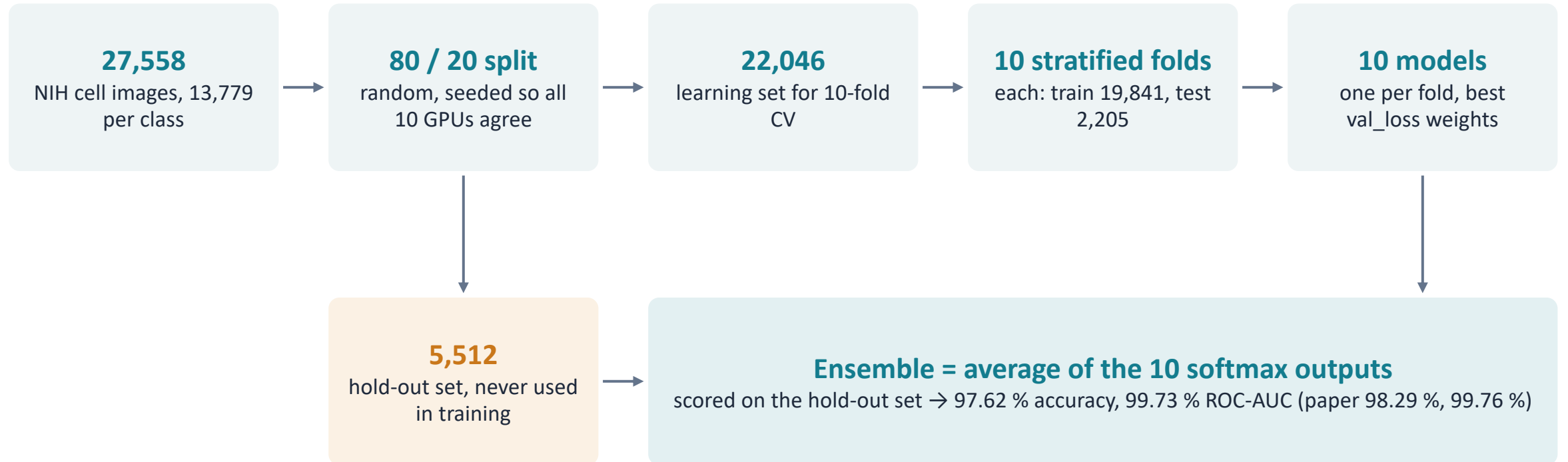
	Pred. parasitized	Pred. uninfected
Parasitized (2,744)	2,682	62 missed
Uninfected (2,768)	32 flagged	2,736

Recall = $2,682 / 2,744 = 97.74 \%$ (text says 98.82 %)

Precision = $2,682 / 2,714 = 98.82 \%$ (text says 97.74 %)

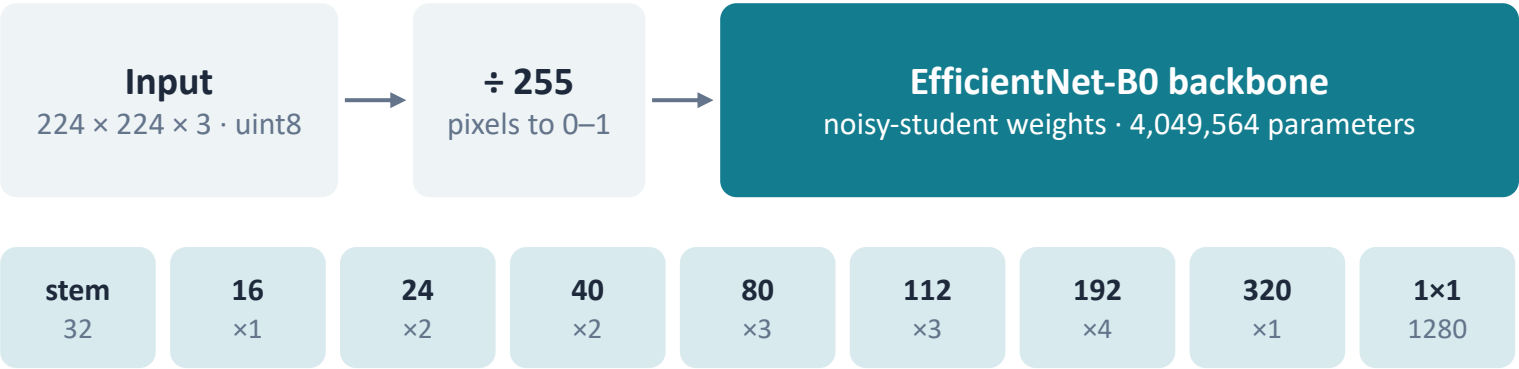
So our first run's "recall 2 pp lower" compared against the wrong number. Against the true 97.74 %, the paper-exact run is 0.67 pp lower.

One split, ten fold models, one ensemble



Every fold model is also scored on its own CV test part (the paper's Table 5) and on the hold-out set (Table 6). Each fold stores a fingerprint of the split (a0cc112e...): all 10 folds match.

EfficientNet-B0 with the paper's dense head



Inside the backbone: a stem convolution, 16 MBConv blocks in 7 stages (output channels, repeats), and a 1×1 convolution $\rightarrow 7 \times 7 \times 1280$ features.



Output: P(parasitized), P(uninfected). Loss: categorical cross-entropy with label smoothing 0.1.

Part	Parameters
EfficientNet-B0 backbone	4,049,564
Dense head	174,370
Total	4,223,934
Trainable	4,181,918
Non-trainable (BatchNorm)	42,016

Identical to the paper's Table 2, digit for digit, checked by an automated test.

Noisy-student weights: EfficientNet-B0 pre-trained on ImageNet plus self-training on unlabelled images (Xie et al., 2020). The paper loads them from the efficientnet package; so do we.

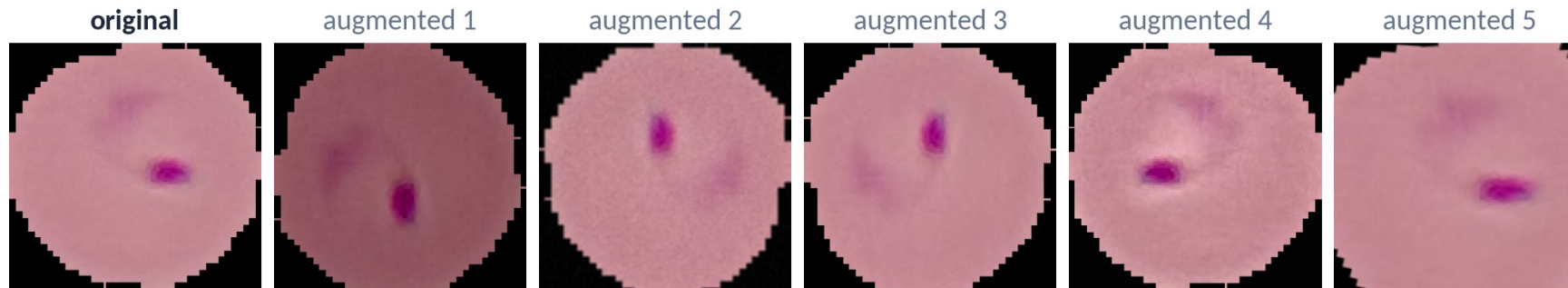
Trained exactly as the authors did

Setting	Value
Optimiser	Adam, learning rate 1e-4
Schedule	halve the LR after 6 epochs without val_loss gain, floor 1e-6
Length	33 epochs × 1,240 steps × 16 images, no early stopping
Checkpoint	keep the weights with the lowest val_loss
Precision	float32 as in the paper (no mixed precision)

AUGMENTATION · a new random version of every image, every epoch

Always: random 90° rotation, flip, transpose

Sometimes: Gaussian noise · motion/median blur · shift-scale-rotate · optical / grid / piecewise-affine distortion · CLAHE, sharpen, emboss, contrast, brightness



How to read the metrics

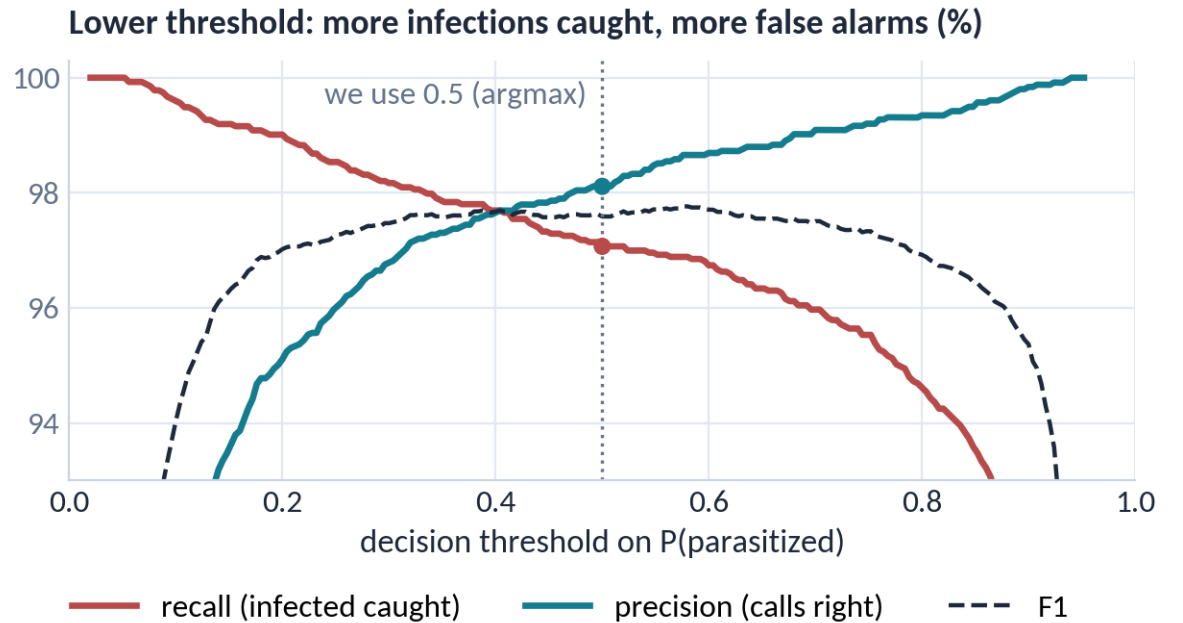
	model says PARASITIZED	model says UNINFECTED
truth: parasitized (2,730)	TP = 2,650 infected, caught	FN = 80 infected, missed
truth: uninfected (2,782)	FP = 51 healthy, false alarm	TN = 2,731 healthy, cleared

Accuracy = (TP + TN) / all = 5,381 / 5,512 = 97.62 %

Recall (sensitivity) = TP / (TP + FN): infected cells caught · 97.07 %

Precision = TP / (TP + FP): 'parasitized' calls that are right · 98.11 %

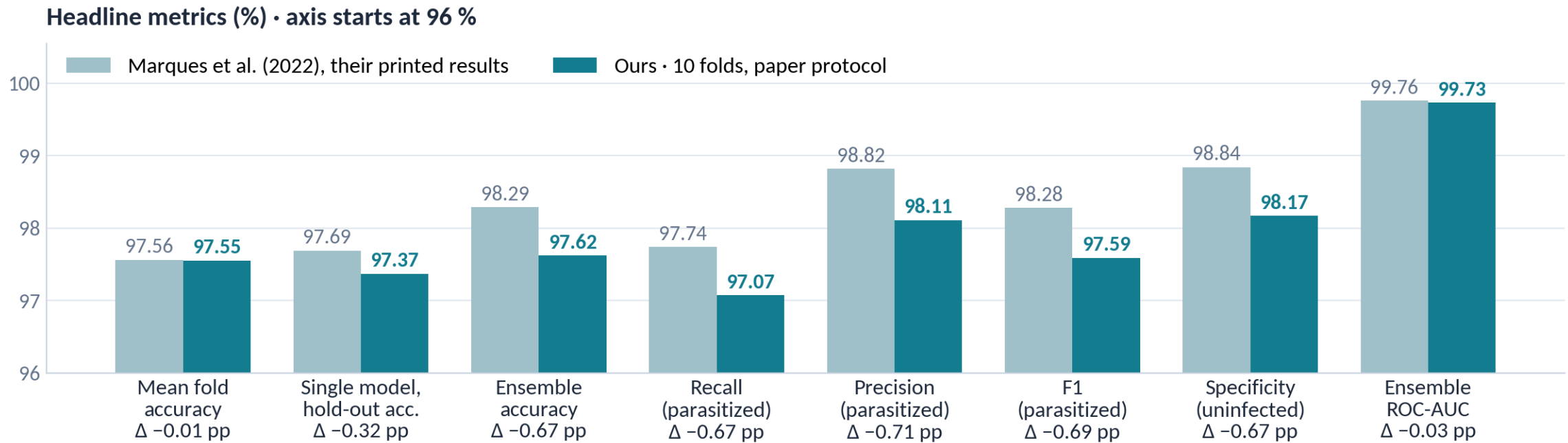
F1 = harmonic mean of precision and recall · 97.59 %



ROC-AUC: chance that an infected cell outscores a healthy one · 99.73 %

pp (percentage point) = plain difference of two percentages: 97.62 – 98.29 = –0.67 pp

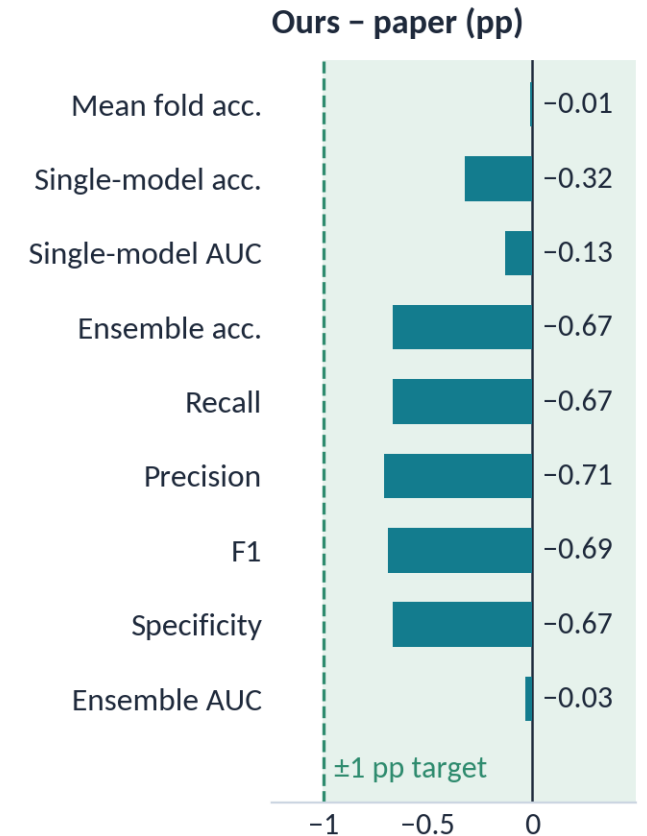
A faithful reproduction: within 1 pp of the paper on every metric



Every metric within 1 pp (src/check_results.py); mean fold accuracy matches to 0.01 pp. Paper protocol = augmented test images, as in the paper's code. On clean images the ensemble scores 97.64 % and the mean fold 97.78 %.

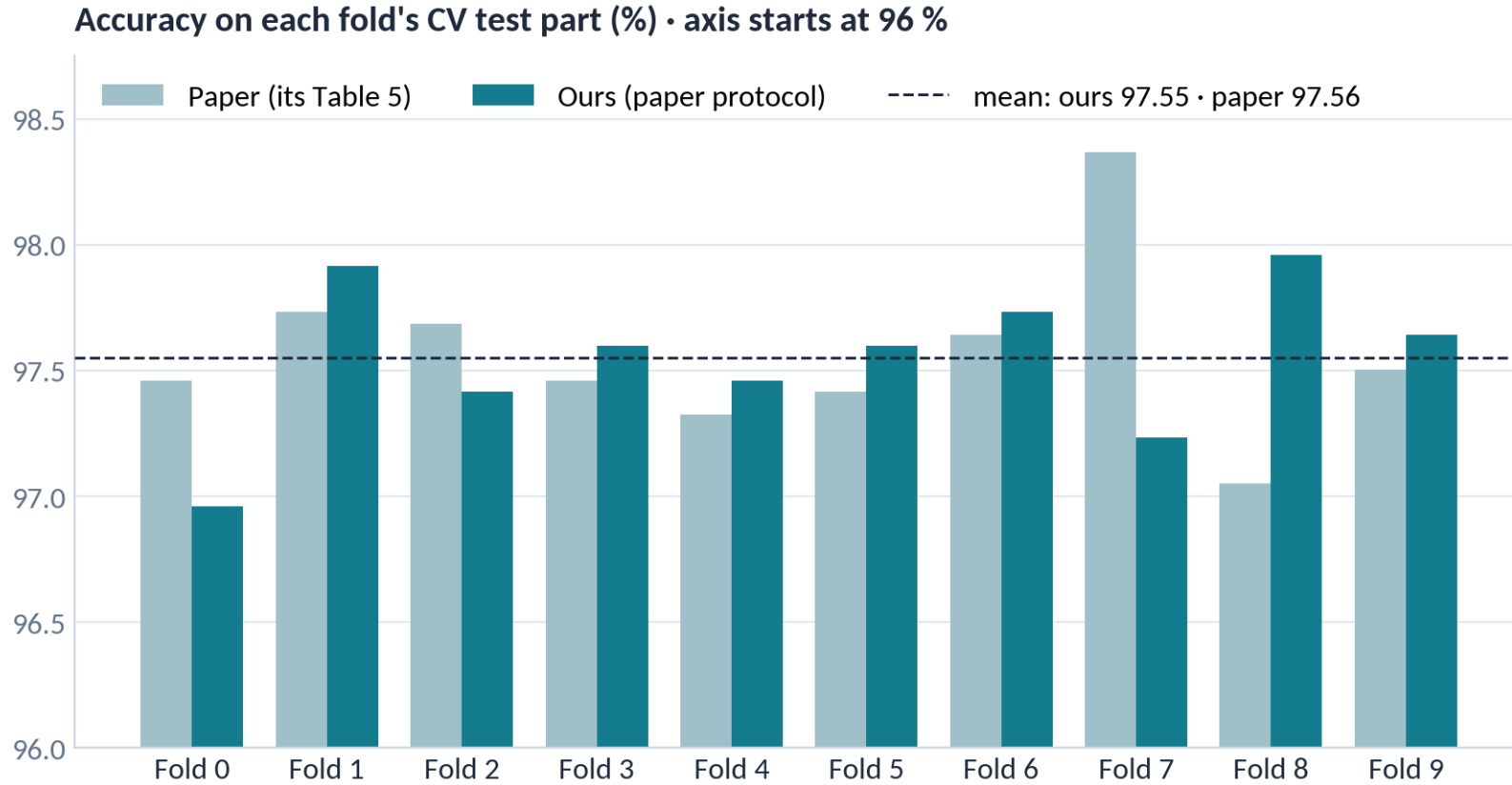
Paper vs. ours on the 5,512-image hold-out set

Metric (%)	Paper	Ours · paper protocol	Ours · clean images	Δ (pp)
Mean CV-fold accuracy (Table 5)	97.56	97.55	97.78	-0.01
Mean single-model hold-out accuracy (Table 6)	97.69	97.37	97.48	-0.32
Mean single-model hold-out ROC-AUC (Table 6)	99.65	99.52	99.52	-0.13
Ensemble accuracy	98.29	97.62	97.64	-0.67
Ensemble recall, parasitized	97.74	97.07	97.18	-0.67
Ensemble precision, parasitized	98.82	98.11	98.04	-0.71
Ensemble F1, parasitized	98.28	97.59	97.61	-0.69
Ensemble specificity (uninfected recall)	98.84	98.17	98.09	-0.67
Ensemble ROC-AUC	99.76	99.73	99.68	-0.03



All nine differences sit inside our ± 1 pp target. The largest, the ensemble's -0.67 pp, is about 3 standard errors on 5,512 cells: the paper's unseeded split and TensorFlow 2.3 vs 2.15 remain different.

Ten folds, one consistent model



± 0.30 pp

spread across folds (SD)

first 3-fold run: ± 0.74 pp

16 – 33

best epoch per fold

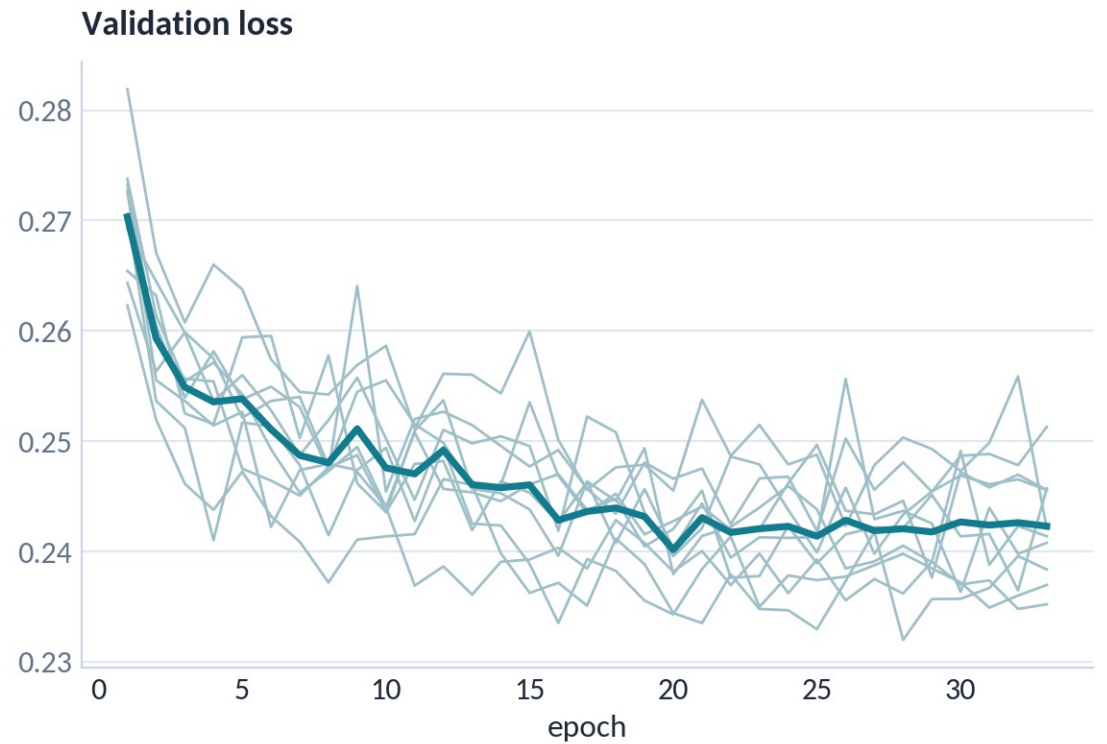
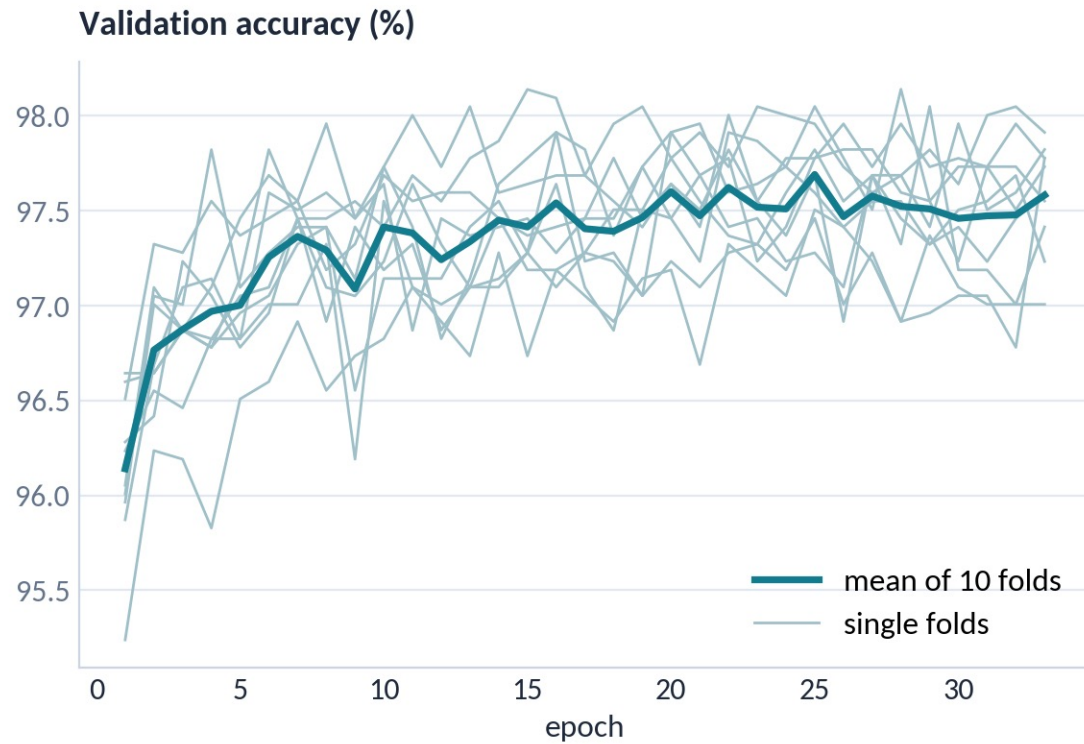
checkpoint = lowest val_loss

$33 \times 1,240$

epochs $\times$ steps, every fold

batch 16, as in the paper

Training behaved like the paper's



Validation = each fold's own CV test part (2,204–2,205 augmented cells). The learning rate halves when val_loss stalls for 6 epochs, and the weights with the lowest val_loss are kept, exactly as in the paper.

Where the 10-model ensemble still errs

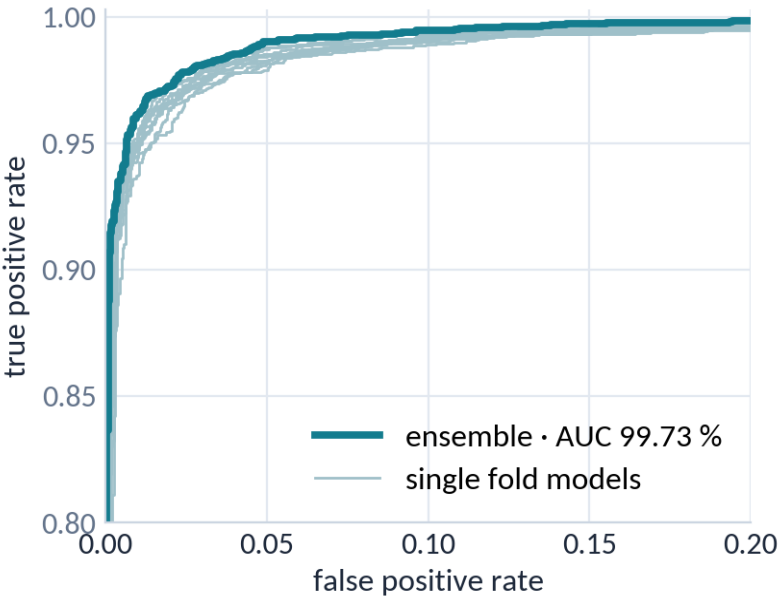
Ours (paper protocol)

	Pred. parasitized	Pred. uninfected
Parasitized	2,650	80
Uninfected	51	2,731

Paper (its printed report)

	Pred. parasitized	Pred. uninfected
Parasitized	2,682	62
Uninfected	32	2,736

ROC on the hold-out set (zoomed)

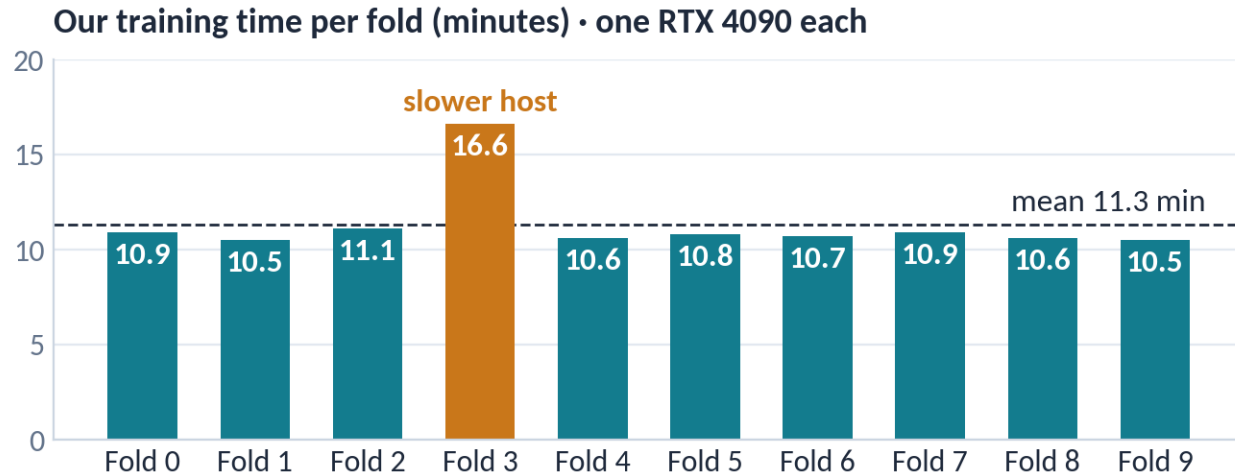


ROC zoomed to the top-left corner: ensemble AUC 99.73 % (paper 99.76 %).



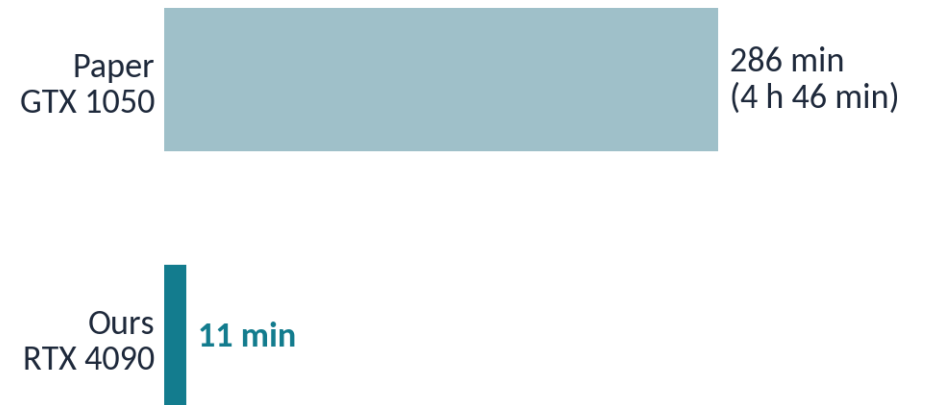
Out of 5,512 hold-out cells: 80 infected cells missed and 51 healthy cells flagged (paper: 62 and 32). For diagnosis a miss costs more than a false alarm, so recall comes first.

Ten GPUs in parallel: about 11 minutes per fold



10 RunPod pods (Secure Cloud, RTX 4090, 16 vCPU each), one fold per pod, all training at the same time. Fold 3 landed on a slower host (16.6 min).

Training time per fold: $\approx 25\times$ faster



$\approx 25\times$
faster per fold

≈ 19 s
per epoch (median)

1 d 23 h
the paper's full run



The 10 trained models are published as a GitHub release, so nobody has to retrain to use or evaluate them.

Five speed-ups, same maths



Decode once

All 27,558 images resized once into a shared 4.1 GB uint8 cache; the paper re-read every PNG every epoch.



Parallel, seeded augmentation

11 worker processes with seeded batches: same result for any worker count. One BLAS thread per worker: 3× throughput.



Bytes to the GPU

Batches cross PCIe as uint8 (4× less data) and are divided by 255 inside the model.



XLA on the training step

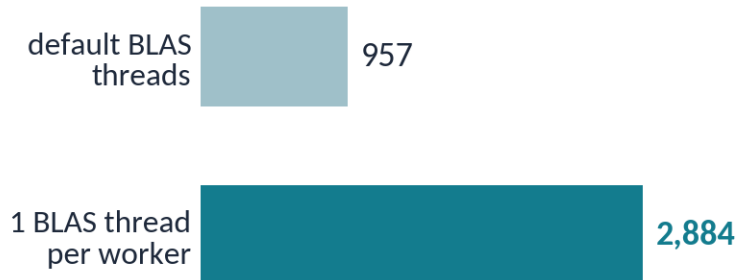
Fused GPU kernels: 21.2 → 12.5 ms per step. Evaluation stays uncompiled, so it never recompiles.



Same numerics, tested

float32 as in the paper (no mixed precision); 32 automated tests pin the recipe.

Augmentation throughput (images/s) · 3.0× more



GPU time per training step (ms) · 1.7× faster



Two proposed changes, one variable at a time



H1 · VALIDATION

Leakage-free, slide-grouped split

CHANGE

Split by slide/patient (StratifiedGroupKFold) instead of random images. Model unchanged.

WE EXPECT

Accuracy and MCC fall below the image-split baseline, and fold-to-fold spread widens.

BECAUSE

Cells from one slide share stain, lighting and the patient's cell shape: a random split lets the model recognise the slide instead of the parasite.



H2 · PREPROCESSING

YUV stain normalisation (Umer [4])

CHANGE

RGB → YUV → histogram-equalise Y → RGB before EfficientNet-B0. Grouped split kept fixed.

WE EXPECT

Higher recall and MCC than the H1 model on unseen slides; little change on the random split.

BECAUSE

Equalising brightness strips slide-specific colour cues, so the network has to rely on parasite shape, the signal that carries across labs.



Falsifiable: if H2's gain over H1 is smaller than the fold-to-fold SD, we reject H2. If grouped and random splits score the same, H1 is rejected and leakage is not a real issue on this dataset.

Ablation plan and success criteria

ID	Split	Preprocessing	What it answers	Status
A0	image-level (paper)	none	Baseline: does our reproduction match the paper?	done ✓ · paper-exact, 10 folds
A1	slide-grouped	none	H1: how big is the leakage gap (A0 – A1)?	next
A2	slide-grouped	YUV + hist-eq	H2: does stain normalisation help on unseen slides (A2 – A1)?	next
A3	image-level	YUV + hist-eq	Control: is H2's gain specific to new slides (A3 – A0 $\approx$ 0)?	next



Metrics

Accuracy, recall, MCC and ROC-AUC: mean $\pm$ SD over the 10 folds plus the hold-out ensemble.



Clinical priority

Recall first: a missed parasitized cell costs more than a false alarm.



Already built

Slide-grouped split and YUV are switches in the code; one 10-fold run takes $\approx$ 11 min per fold on RunPod.

CONCLUSION

Where we stand



Reproduced

The paper's recipe, rebuilt from its own code, lands within 1 pp on every metric (mean fold 97.55 vs 97.56 %).



Corrected

The paper evaluates on augmented images and swaps precision and recall in its text; we report both views.



Fast and shareable

10 folds in parallel, ≈ 11 min each instead of ≈ 4 h 46 min; the trained models are published.



Next

A1–A3 test whether 97–98 % holds on unseen slides (H1) and whether stain normalisation helps (H2).

Thank you · questions welcome

github.com/amathziah/malaria-detection

The eleven reviewed studies

- [1] Z. Liang et al., "CNN-Based Image Analysis for Malaria Diagnosis," IEEE BIBM, 2016.
- [2] S. Rajaraman et al., "Pre-trained CNNs as feature extractors toward improved malaria parasite detection," PeerJ 6:e4568, 2018.
- [3] S. Rajaraman et al., "Performance evaluation of deep neural ensembles toward malaria parasite detection," PeerJ 7:e6977, 2019.
- [4] M. Umer et al., "A Novel Stacked CNN for Malarial Parasite Detection in Thin Blood Smear Images," IEEE Access 8, 2020.
- [5] K. M. F. Fuhad et al., "Deep Learning Based Automatic Malaria Parasite Detection ... Smartphone Application," Diagnostics 10(5):329, 2020.
- [6] O. S. Zhao et al., "Convolutional neural networks to automate the screening of malaria in low-resource countries," PeerJ 8:e9674, 2020.
- [7] M. Masud et al., "Leveraging Deep Learning Techniques for Malaria Parasite Detection Using Mobile Application," WCMC 2020:8895429.
- [8] G. Marques et al., "An ensemble-based approach for automated medical diagnosis of malaria using EfficientNet," MTAP 81, 2022 (base paper; code: Springer supplementary file MOESM1).
- [9] G. Madhu et al., "Intelligent diagnostic model ... imperative inception-based capsule neural networks," Scientific Reports 13, 2023.
- [10] K. Hemachandran et al., "Performance Analysis of Deep Learning Algorithms in Diagnosis of Malaria Disease," Diagnostics 13(3):534, 2023.
- [11] M. Mujahid et al., "Efficient deep learning-based approach for malaria detection using red blood cell smears," Scientific Reports 14, 2024.